Systemsoftware – Lösungsblatt 1

Wintersemester 2026/27

Name: Paul Lichtenhagen **Matrikelnummer: 1778614**

Aufgabe 1: Prozess, Thread und Programm

Programm: Ausführbarer Code, liegt auf SSD / Festplatte. Kann mehrfach gestartet werden und dabei mehrere Prozesse erzeugen. Beispiele: .exe-Datei , python-Skript.

Prozess: Laufende Instanz eines Programms. Jeder Prozess besitzt einen eigenen Adressraum und hat >= 1 Thread(s). Beispiele: Terminal, Webbrowser.

Thread: Ausführungseinheit innerhalb eines Prozesses. Threads teilen sich den Adressraum des Prozesses und können parallel ausgeführt werden. Beispiele: Haupt-Thread eines Programms, Worker-Threads in einem Webserver.

Aufgabe 2: Parallelisierung und Speedup

a) Grenzen der Beschleunigung

1. Amdahlsches Gesetz: $S = \frac{1}{(1-P)+\frac{P}{N}}$ Nicht jeder Teil eines Programms kann parallel ausgeführt werden. Der sequenzielle Anteil muss weiterhin auf einem einzelnen Kern ausgeführt werden.

2. Begrenzte Taktfrequenz Eine Erhöhung der Taktfrequenz führt zu einer höheren Leistungsaufnahme und damit zu mehr Abwärme. Dies limitiert die maximale Taktfrequenz, die praktisch erreicht werden kann.

3. Pipeline-Stalls

Eine Möglichkeit, höhere Taktfrequenzen zu erreichen, besteht darin, die Prozessor-Pipeline in mehr, kürzere Stufen aufzuteilen. Dies kann jedoch zu Pipeline-Stalls führen, wenn Abhängigkeiten zwischen aufeinanderfolgenden Befehlen auftreten.

b) Amdahlsches Gesetz

Beispielrechnung:

$$S(2) = \frac{1}{(1-0{,}25)+\frac{0{,}25}{2}} = \frac{1}{0{,}75+0{,}125} = \frac{1}{0{,}875} = \frac{8}{7} \approx \boxed{1{,}14}.$$

Parallelisierbarer Anteil	2 Kerne	4 Kerne	8 Kerne	16 Kerne
25 %	1,14	1,23	1,28	1,31
50 %	1,33	1,60	1,78	1,88
75 %	1,60	2,29	2,91	3,37

Aufgabe 3: Line Counter

3) Dokumentation des Experimentierens

Versuchsaufbau und Methodik

System: Windows, OpenJDK 21.0.8, 16 logische Prozessoren.

Testdaten: 256 `.java`-Dateien (256 MiB, etwa 4,19 Mio. Zeilen).

Ausführung: Parallelisiert mit 1, 2, 4, 8, 16 und 32 Threads.

Aufrufe und Messergebnisse

Die PowerShell-Aufrufe hatten folgende Form: `$benchRoot` bezeichnet den Ordner mit der kompilierten Klasse und `$benchData` den Ordner mit den Testdateien:

```
java -cp $benchRoot Counter $benchData '.*\.java' 1
java -cp $benchRoot Counter $benchData '.*\.java' 2
java -cp $benchRoot Counter $benchData '.*\.java' 4
java -cp $benchRoot Counter $benchData '.*\.java' 8
java -cp $benchRoot Counter $benchData '.*\.java' 16
java -cp $benchRoot Counter $benchData '.*\.java' 32
```

Anzahl Threads	Laufzeit: Median (ms)
1	282,9
2	179,6
4	131,5
8	118,9
16	124,4
32	122,7

Beobachtungen

Bis zu acht Threads nahm die Laufzeit ab. Gegenüber der Ausführung mit einem Thread ergab sich bei acht Threads ein Speedup von

$$S(8) = \frac{282{,}9}{118{,}9} \approx 2{,}38.$$